Informe

Solución Informática
Programación Orientada a Objeto Seguro

CASO	:	Cine
Asignatura	:	Programación Orientada a Objeto Seguro Evaluación Sumativa N.° 1 (versión corregida)
Docente	:	Michael Alexis Arjel Mayerovich
Sección	:	2026/P TI3V21/114-2A-F2/V Puente Alto FB5.
Integrantes	:	Fernando A. Durán Leiva. Johan Patrichs Vargas Catalán

Tabla de Contenidos.

Tabla de Contenidos. 2
1. El problema en sus palabras. 3
2. Diagrama de clases. 4
3. Explicación de cada clase. 7
4. Justificación de las relaciones. 10
5. Cobertura de los seis requisitos mínimos. 12
6. Correcciones respecto de la entrega original. 13
Anexo 1 - Requerimiento Inicial. 14
Anexo 2 - Requerimientos de Profundización en las Especificaciones del Cliente. 15
Anexo 3 - Respuesta a Profundización en las Especificaciones. 15

1. El problema en sus palabras.

Para la actividad se reservó el negocio **11 – Cine**. El cine necesita un sistema para vender, en sus cajas, entradas y productos de dulcería. En una misma compra el cliente suele llevar varias entradas y varios productos, y todo debe quedar como una sola venta con sus líneas de detalle. Las cajas están separadas: el boletero solo vende entradas y el encargado de dulcería solo vende snacks y bebidas. El precio de la entrada depende del tipo de función (2D, 3D o VIP; la VIP además tiene asientos limitados), y la venta se debe impedir si la función ya no tiene asientos o si la película tiene una edad mínima que el comprador no cumple, dato que se valida a partir de su RUT y su edad. Los productos importados se cotizan con el valor del dólar del día.

Las reglas del proceso que el sistema debe cumplir son:

1. El sistema debe diferenciar tres tipos de función: 2D, 3D y VIP.
2. Cada tipo de función tiene un precio distinto y asientos limitados; la VIP tiene además un cupo propio de asientos.
3. Existen dos tipos de trabajador con permisos distintos: el boletero, que solo vende entradas, y el encargado de dulcería, que solo vende snacks y bebidas. Atienden cajas separadas.
4. Antes de vender una entrada para una película con clasificación restringida se valida el RUT y la edad del comprador.
5. Antes de vender una entrada se controla que la función tenga asientos disponibles.
6. Una compra con varias entradas y productos se registra como una sola venta con líneas de detalle.
7. Los productos importados de dulcería actualizan su precio según el valor diario del dólar.

2. Diagrama de clases.

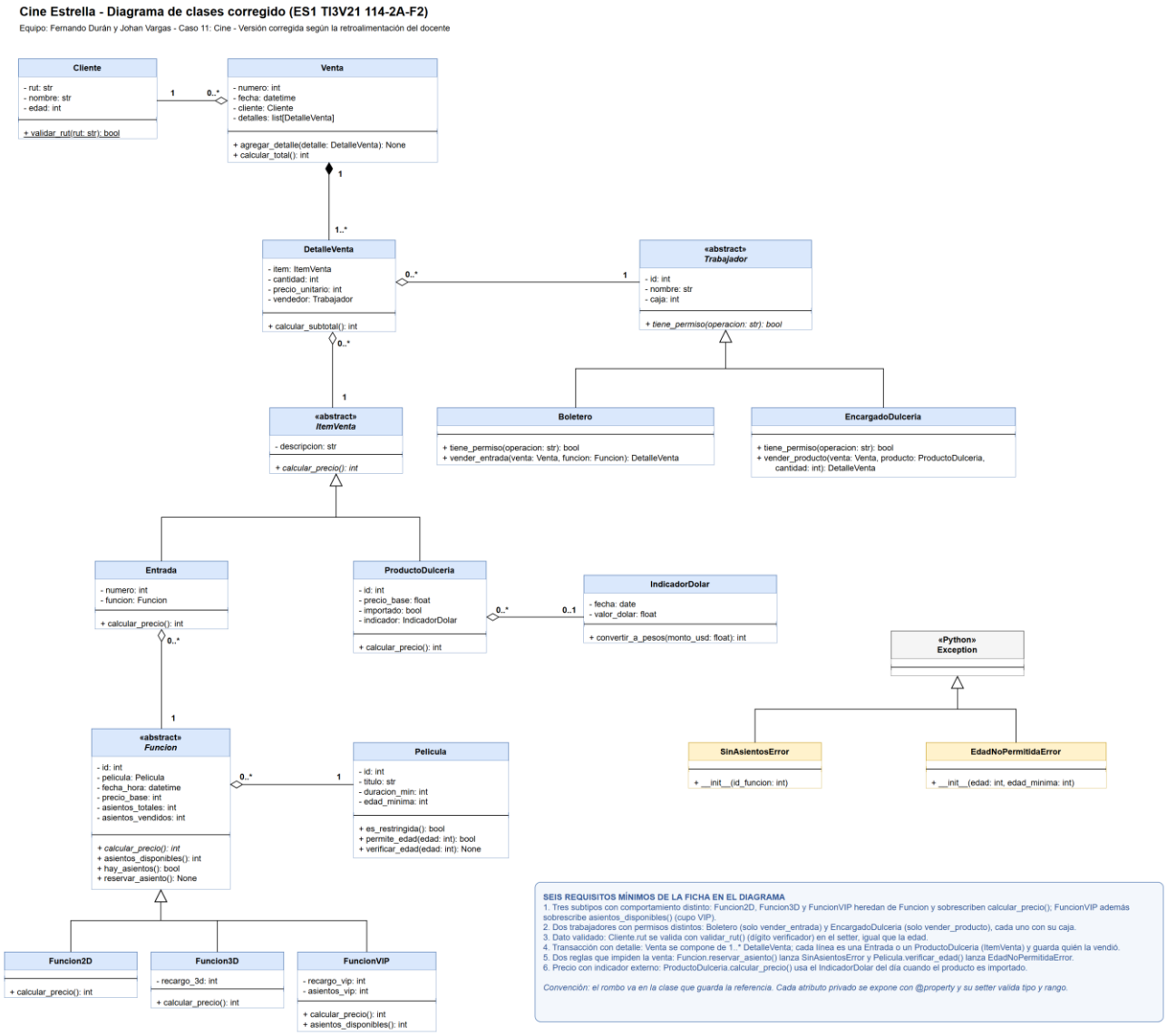
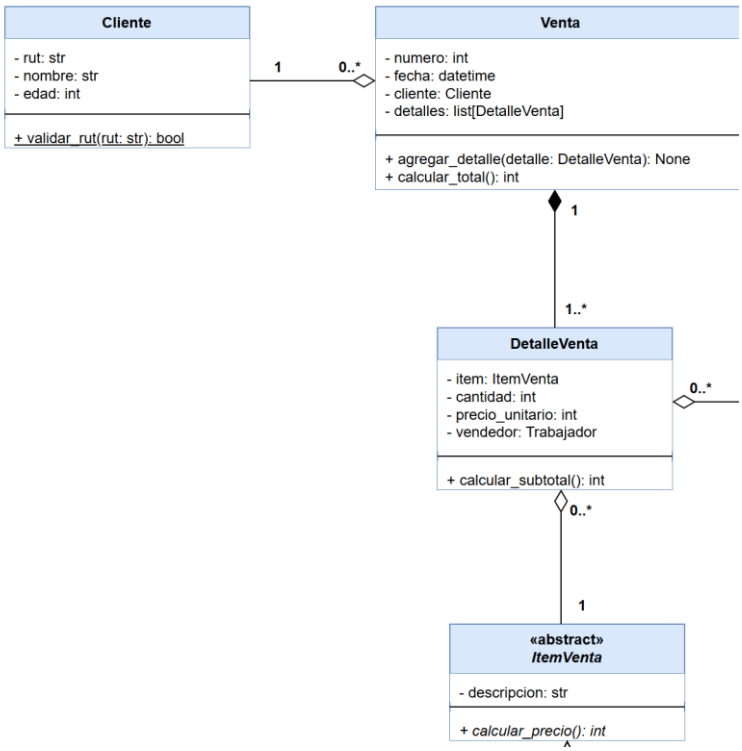


Figura 1. Diagrama de clases corregido. Archivo editable: 11-Cine-Fernando-Duran-Johan-Vargas-corregido.drawio. En las páginas siguientes se muestra ampliado por zonas.

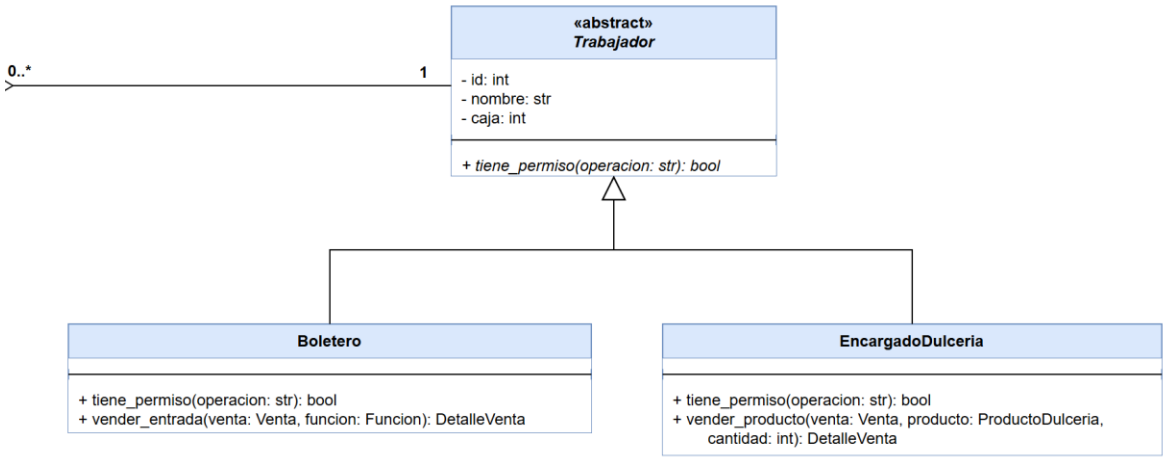
El diagrama muestra todas las clases del sistema con sus tres secciones (nombre, atributos y métodos), la visibilidad de cada miembro (- privado, + público), el tipo de cada atributo y los parámetros y el retorno de cada método. Las clases abstractas llevan el estereotipo «abstract» y su nombre en cursiva; los métodos abstractos van en cursiva y el método estático validar_rut() va subrayado. Los tipos se escriben como en Python (str, int, float, bool), porque el sistema se construye en ese lenguaje. En las agregaciones y en la composición, el rombo se dibuja en la clase que guarda la referencia a la otra.

Vista ampliada por zonas

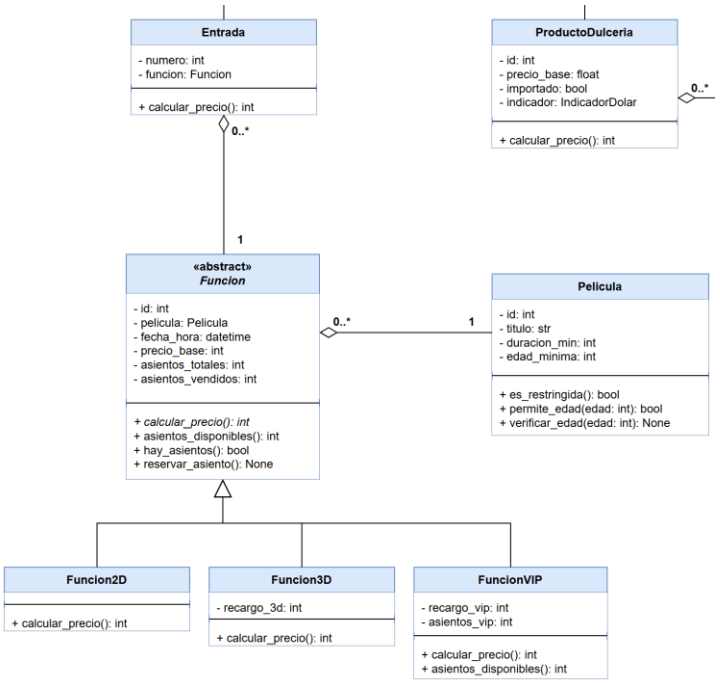
Para que cada clase se lea con claridad, el mismo diagrama se presenta ampliado en cinco zonas.



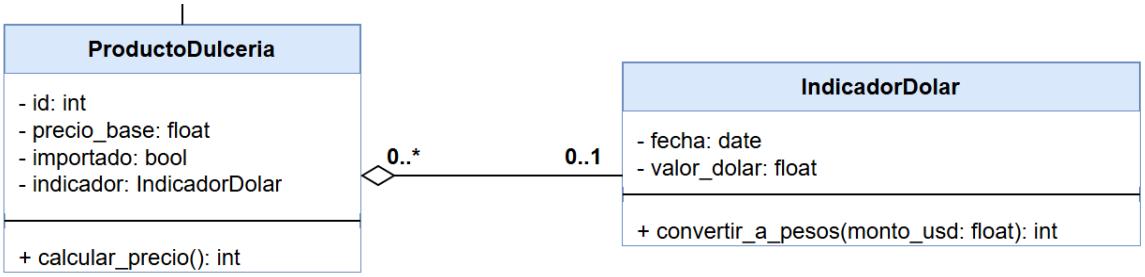
Zona 1: venta, detalle, ítems y cliente.



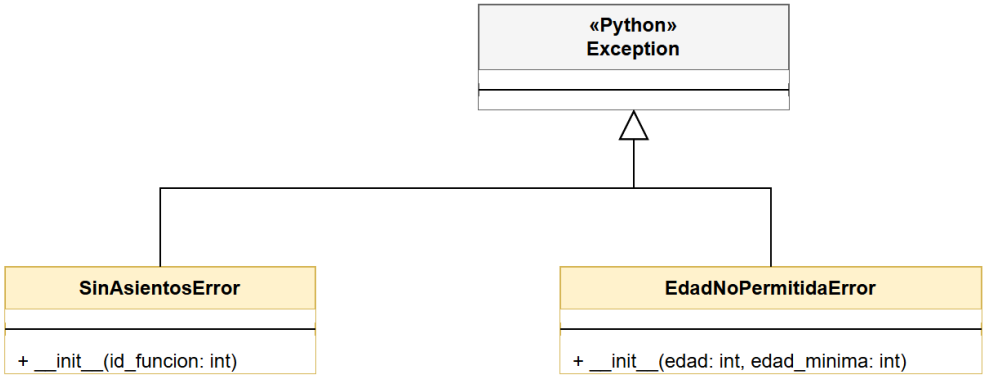
Zona 2: trabajadores y permisos.



Zona 3: entradas, funciones, películas y subtipos de función.



Zona 4: dulcería e indicador del dólar.



Zona 5: excepciones de las reglas del negocio.

3. Explicación de cada clase.

Clases candidatas y decisiones de modelado

Las clases se identificaron a partir de los sustantivos del enunciado que tienen datos y comportamiento propios: cliente, venta, línea de venta, entrada, función, película, producto, trabajador e indicador del dólar. Respecto del análisis inicial se tomaron tres decisiones:

- **ProductoNacional y ProductoImportado** se reemplazaron por el atributo `importado: bool` de ProductoDulceria. La única diferencia entre ambos era si el precio se convierte con el dólar: no tenían atributos ni métodos propios distintos. Mantenerlos como subclases habría sido sobre-modelar; la herencia con comportamiento distinto que exige la ficha está en Funcion.
- **TipoCambioDolar** pasó a llamarse **IndicadorDolar**, porque representa el indicador externo que entrega la API (fecha y valor del dólar observado), y se le agregó el método `convertir_a_pesos()`.
- Se agregó la clase abstracta **ItemVenta**, de la que heredan Entrada y ProductoDulceria, para que una misma línea de venta pueda contener cualquiera de los dos.

Descripción de las clases

Cliente

Persona que compra en el cine. Existe porque la ficha exige un dato validado y un control de edad: guarda el RUT, el nombre y la edad. El método estático `validar_rut()` comprueba el formato y el dígito verificador (módulo 11), y el setter del RUT rechaza cualquier valor inválido.

Venta

Transacción completa de un cliente en una visita. Reúne en un solo registro las entradas y los productos que compra, con su fecha y su cliente. `agregar_detalle()` suma una línea y `calcular_total()` suma los subtotales de todas sus líneas.

DetalleVenta

Cada línea de una venta: qué se vendió (un ItemVenta), cuántas unidades, a qué precio unitario y qué trabajador lo vendió. Guarda el precio del momento porque el dólar cambia a diario y una venta ya hecha debe conservar su total. `calcular_subtotal()` multiplica cantidad por precio unitario.

ItemVenta (abstracta)

Lo que se puede vender en una línea. Existe para que DetalleVenta trate igual a una entrada y a un producto: ambos responden a `calcular_precio()`, cada uno a su manera.

Entrada

Ticket de acceso a una función. Su precio es el de la función, por lo que cambia según sea 2D, 3D o VIP.

ProductoDulceria

Snack o bebida de la dulcería. Si es importado, su precio base está en dólares y `calcular_precio()` lo convierte a pesos con el IndicadorDolar del día; si es nacional, su precio base ya está en pesos.

IndicadorDolar

Valor del dólar observado de un día, obtenido desde una API pública. Existe porque la ficha exige un precio que dependa de un indicador externo. `convertir_a_pesos()` hace la conversión.

Pelicula

Obra que exhibe el cine, con su título, duración y edad mínima. `es_restringida()` indica si tiene edad mínima, `permite_edad()` la compara con la del comprador y `verificar_edad()` bloquea la venta lanzando EdadNoPermitidaError.

Funcion (abstracta)

Exhibición de una película en una fecha y hora, con precio base y capacidad de la sala. Controla los asientos: `reservar_asiento()` ocupa uno y lanza SinAsientosError cuando no quedan. Es abstracta porque la forma de calcular el precio depende del formato.

Funcion2D

Función normal. `calcular_precio()` devuelve el precio base.

Funcion3D

Función con tecnología 3D. `calcular_precio()` suma al precio base el recargo 3D.

FuncionVIP

Función premium. `calcular_precio()` suma el recargo VIP y `asientos_disponibles()` solo cuenta su cupo de asientos VIP, porque el cliente indicó que la VIP tiene asientos limitados.

Trabajador (abstracta)

Empleado que atiende una caja del cine (identificador, nombre y número de caja). Declara `tiene_permiso()`, que cada cargo responde de forma distinta.

Boletero

Atiende la boletería y solo puede vender entradas. `vender_entrada()` aplica las dos reglas del negocio (edad y asientos) antes de agregar la entrada a la venta.

EncargadoDulceria

Atiende la dulcería y solo puede vender productos. `vender_producto()` agrega la línea con el precio del día.

SinAsientosError

Excepción de la regla «no vender una entrada si la función no tiene asientos». La lanza `Funcion.reservar_asiento()`.

EdadNoPermitidaError

Excepción de la regla «no vender una entrada a quien no cumple la edad mínima de la película». La lanza `Pelicula.verificar_edad()`.

4. Justificación de las relaciones.

Cada relación se decidió con el árbol de dos preguntas visto en clases: primero, ¿un objeto es un objeto del otro tipo? Si la respuesta es sí, corresponde herencia. Si no, ¿el lado «muchos» acepta cero? Si lo acepta, la parte existe sin el todo y corresponde agregación; si exige al menos uno, corresponde composición. Todas las asociaciones llevan multiplicidad en ambos extremos.

Herencias (triángulo hueco)

Funcion → Funcion2D, Funcion3D, FuncionVIP

Una función 2D **es una** función, igual que una 3D y una VIP. Comparten película, fecha, precio base y asientos, y se diferencian en cómo calculan el precio: cada subtipo sobrescribe `calcular_precio()`, y FuncionVIP también `asientos_disponibles()`. Esto reemplaza al atributo `tipo_formato: str` de la entrega anterior y es el polimorfismo que exige la ficha.

Trabajador → Boletero, EncargadoDulceria

Un boletero **es un** trabajador y un encargado de dulcería también. Comparten identificador, nombre y caja; cada uno responde distinto a `tiene_permiso()` y solo tiene el método de venta que le corresponde, lo que modela las cajas separadas.

ItemVenta → Entrada, ProductoDulceria

Una entrada **es un** ítem de venta y un producto de dulcería también. Ambos tienen descripción y calculan su precio de forma distinta: la entrada según su función y el producto según su origen y el dólar del día.

Exception → SinAsientosError, EdadNoPermitidaError

Cada excepción **es una** excepción de Python. Tener una clase propia por regla permite capturar cada bloqueo por separado e informar el motivo exacto.

Composición (rombo relleno)

Venta 1 ◆— 1..* DetalleVenta

Una venta no es un tipo de detalle. El lado «muchos» no acepta cero: una venta sin líneas no es una venta, por eso la multiplicidad es 1..* y no se guarda una venta vacía. Además, cada línea pertenece a una sola venta y no tiene sentido sin ella: si la venta desaparece, sus líneas también. Corresponde composición.

Agregaciones (rombo hueco)

Venta 0..* ◇— 1 Cliente

Cada venta tiene exactamente un cliente. Un cliente puede estar registrado sin haber comprado, así que tiene cero o muchas ventas: el lado «muchos» acepta cero.

DetalleVenta 0..* ◇— 1 ItemVenta

Cada línea vende exactamente un ítem. Un producto puede aparecer en cero o muchas líneas (un producto recién creado todavía no se ha vendido).

DetalleVenta 0..* ◇— 1 Trabajador

Cada línea la registra un trabajador, y un trabajador puede no haber vendido nada todavía. La relación va en la línea y no en la venta porque, según la respuesta 13 del cliente (Anexo 3), en una misma compra las entradas las vende el boletero y los productos el encargado, cada uno en su caja.

Entrada 0..* ◇— 1 Funcion

Una entrada pertenece a una sola función, y una función puede no tener entradas vendidas. En la entrega anterior esta relación estaba con línea punteada y sin multiplicidad.

Funcion 0..* ◇— 1 Pelicula

Una función exhibe una sola película, y una película puede existir sin funciones programadas. También estaba con línea punteada y sin multiplicidad.

ProductoDulceria 0..* ◇— 0..1 IndicadorDolar

Un producto importado usa el dólar del día y uno nacional no lo usa (por eso 0..1). Un mismo indicador sirve para cotizar muchos productos, o ninguno.

Resumen

Relación	Tipo	Multiplicidad	Pregunta que decide
Funcion → 2D, 3D, VIP	Herencia	—	¿Es un? Sí
Trabajador → Boletero, EncargadoDulceria	Herencia	—	¿Es un? Sí
ItemVenta → Entrada, ProductoDulceria	Herencia	—	¿Es un? Sí
Exception → SinAsientosError, EdadNoPermitidaError	Herencia	—	¿Es un? Sí
Venta — DetalleVenta	Composición	1 a 1..*	No acepta cero
Venta — Cliente	Agregación	0..* a 1	Acepta cero
DetalleVenta — ItemVenta	Agregación	0..* a 1	Acepta cero

Relación	Tipo	Multiplicidad	Pregunta que decide
DetalleVenta — Trabajador	Agregación	0..* a 1	Acepta cero
Entrada — Funcion	Agregación	0..* a 1	Acepta cero
Funcion — Pelicula	Agregación	0..* a 1	Acepta cero
ProductoDulceria — IndicadorDolar	Agregación	0..* a 0..1	Acepta cero

5. Cobertura de los seis requisitos mínimos.

La tabla muestra qué parte del diagrama sostiene cada requisito de la ficha del negocio.

Requisito de la ficha	Cómo lo sostiene el diagrama
1. Tres subtipos con un método que se comporta distinto	Funcion (abstracta) con Funcion2D, Funcion3D y FuncionVIP, que sobrescriben calcular_precio(). FuncionVIP también sobrescribe asientos_disponibles().
2. Dos tipos de trabajador con permisos distintos	Trabajador (abstracta) con Boletero (solo vender_entrada()) y EncargadoDulceria (solo vender_producto()); tiene_permiso() responde distinto en cada uno. Cada trabajador tiene su caja.
3. Al menos un dato con validación	Cliente.rut, validado con validar_rut() (formato y dígito verificador) en su setter. También se valida la edad.
4. Una transacción con líneas de detalle	Venta compuesta por 1..* DetalleVenta; cada línea tiene un ItemVenta (Entrada o ProductoDulceria), cantidad, precio unitario y vendedor.
5. Dos reglas que impiden una operación	Funcion.reservar_asiento() lanza SinAsientosError; Pelicula.verificar_edad() lanza EdadNoPermitidaError. Boletero.vender_entrada() aplica ambas antes de agregar la entrada.
6. Un precio que depende de un indicador externo	ProductoDulceria.calcular_precio() usa IndicadorDolar.convertir_a_pesos() cuando el producto es importado. El valor del dólar se obtiene de una API pública.

6. Correcciones respecto de la entrega original.

Esta versión incorpora la retroalimentación del docente sobre la primera entrega. Cada observación y su corrección:

Observación del docente	Corrección aplicada
De los seis requisitos de la ficha, el diagrama sostenía tres; los otros tres estaban en el informe pero no en el dibujo.	El diagrama sostiene ahora los seis requisitos (sección 5): subtipos de Funcion, Cliente con validar_rut() y Venta compuesta por DetalleVenta se suman a los trabajadores, las dos reglas y el precio con el dólar.
Funcion resolvía el tipo con un atributo tipo_formato: str, sin herencia ni polimorfismo.	Se eliminó tipo_formato. Funcion es abstracta y Funcion2D, Funcion3D y FuncionVIP heredan de ella y sobrescriben calcular_precio().
No había DetalleVenta: Venta se conectaba directo a Entrada y a ProductoDulceria.	Se agregó DetalleVenta (composición Venta 1 a 1..* DetalleVenta) y la clase abstracta ItemVenta, de la que heredan Entrada y ProductoDulceria.
No había Cliente ni validación de RUT en el diagrama.	Se agregó Cliente (rut, nombre, edad) con validar_rut(), relacionado con Venta (0..* a 1).
El diagrama incluía objetos instanciados (Producto_1, Pelicula_2 y otros): era un diagrama de objetos.	Se eliminaron todos los objetos. El diagrama contiene solo clases, cada una con sus tres secciones.
Entrada-Funcion y Funcion-Pelicula quedaron con línea punteada y sin multiplicidad.	Ambas son agregaciones con línea continua, rombo hueco y multiplicidad en los dos extremos (Entrada 0..* a 1 Funcion; Funcion 0..* a 1 Pelicula).
El archivo incluía una imagen de portada de otro documento de INACAP del año 2014.	Se quitó esa imagen de la portada.
(Ajuste propio) El diagrama estaba al final del informe.	El informe sigue el orden de la pauta: problema, diagrama, explicación de cada clase y justificación de las relaciones.

Anexo 1 - Requerimiento Inicial.

Tenemos tres tipos de función: 2D, 3D y VIP. Cada tipo tiene un precio de entrada distinto, y VIP además tiene asientos limitados, así que el sistema debe calcular el precio de la entrada según el tipo de función.

Trabajan dos roles: el boletero, que vende entradas, y el encargado de dulcería, que vende confites y bebidas, pero no puede vender entradas ni el boletero puede vender dulcería (son cajas separadas).

Para películas con clasificación restringida pedimos el RUT o la edad del comprador, y el sistema debe validar ese dato antes de vender.

En la misma compra el cliente suele llevar varias entradas y productos de dulcería juntos, y todo eso debe quedar como detalle de una misma venta.

No queremos vender una entrada si la función ya no tiene asientos disponibles, ni vender una entrada para una película clasificada a alguien menor de la edad mínima permitida.

Los dulces y snacks importados que vendemos en dulcería se cotizan según el dólar del día.

Saludos,

Michael Alexis Arjel Mayerovich
Docente
INACAP Sede Puente Alto
Av. Concha y Toro 2730
www.inacap.cl

Anexo 2 - Requerimientos de Profundización en las Especificaciones del Cliente.

Para una implementación real del sistema es necesario proveer las siguientes definiciones adicionales:

1. Definir los valores exactos de precio para funciones 2D, 3D y VIP.
2. Definir el número de salas de exhibición que contemplan.
3. Determinar la capacidad máxima de asientos por sala.
4. Especificar si existe numeración de asientos o asignación automática.
5. Definir todas las categorías de clasificación de películas (ATP, +14, +18, etc.).
6. La planilla de trabajadores del cine, ¿contemplará más personas, administrativo, seguridad, etc.?
7. Determinar si una venta puede ser anulada o modificada posteriormente.
8. Definir los canales de venta para las entradas (boletería, internet, etc.).
9. Definir medios de pago aceptados.
10. Especificar la frecuencia de actualización del valor del dólar.
11. Indicar si los productos importados representan todo el producto o solo una parte de su costo.
12. Definir si el sistema manejará descuentos, promociones o programas de fidelización.
13. Determinar si las ventas de boletería y dulcería pueden ser atendidas simultáneamente por trabajadores distintos dentro de la misma transacción.

Anexo 3 - Respuesta a Profundización en las Especificaciones.

Estimados Fernando y Johan,

Les respondo por punto:

.

1. Precios de 2D, 3D y VIP: no tengo valores fijos, pueden asumir montos razonables para su propuesta. Lo que me importa es que el sistema calcule el precio según el tipo de función, no el monto exacto en pesos.
2. Número de salas: no es un dato que necesiten fijar, el sistema debe funcionar igual con una sala o con diez.
3. Capacidad máxima de asientos por sala: mismo caso, asuman un número que les sirva para probar el sistema. Lo que me importa es que no se vendan más entradas que asientos disponibles.
4. Numeración de asientos o asignación automática: eso lo dejo a su criterio de diseño, cualquiera de las dos formas me sirve mientras se respete el límite de VIP.

5. Categorías de clasificación: no necesito el listado oficial completo. Con que el sistema distinga si una película tiene clasificación restringida (y su edad mínima asociada) o no, es suficiente.
6. Planilla de trabajadores: por ahora son solo los dos roles que les mencioné, boletero y encargado de dulcería. No contemplamos administrativos ni seguridad en el sistema.
7. Anulación o modificación de una venta: no, por ahora no necesitamos eso.
8. Canales de venta: la venta es presencial, en las cajas del cine. No vendemos por internet.
9. Medios de pago: no es relevante para el sistema que están construyendo, no necesito que lo modelen.
10. Frecuencia de actualización del dólar: se actualiza a diario, con el valor del día.
11. Productos importados, todo o parte del costo: el precio final del producto depende del valor del dólar del día, la fórmula exacta se las dejo a ustedes.
12. Descuentos, promociones o fidelización: no, no vamos a manejar eso en esta primera versión.
13. Ventas simultáneas en boletería y dulcería: sí, exactamente como les dije, son cajas separadas y cada trabajador atiende la suya en paralelo.